

FORMATION EMBARQUÉE

TP — tp2 vhdI uart

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 2 — Chaîne UART complète en VHDL (≈ 8 h, en 3 séances)

Objectif pédagogique : concevoir, simuler et valider un périphérique matériel complet — émetteur + récepteur UART avec anti-métastabilité et sur-échantillonnage — en appliquant la méthode professionnelle : spécification → schéma → code → testbench → chronogramme.

 **Fiches minutées séance par séance** : [Séance 1](#) · [Séance 2](#) · [Séance 3](#). Cette page est la vue d'ensemble et la grille de notation.

Outils

- **GHDL + GTKWave** (gratuits) : `sudo apt install ghdl gtkwave` ou <https://www.edaplayground.com> (rien à installer).
- Aucune carte FPGA nécessaire ; une section finale optionnelle donne les contraintes pour Basys 3 / Tang Nano.

Script de travail (à créer une fois, `simule.sh`) :

```
#!/bin/sh
set -e
ghdl -a --std=08 "$@"          # analyse des fichiers passés en argument
TB=$(basename "${!#}" .vhd)    # dernier fichier = testbench
ghdl -e --std=08 "$TB"
ghdl -r --std=08 "$TB" --wave="$TB.ghw" --stop-time=5ms
gtkwave "$TB.ghw" &
```

Séance 1 (2 h) — Spécification et émetteur

Étape 1.1 — Écrire la spécification AVANT le code

Recopie et complète ce tableau (c'est un livrable) :

Paramètre	Valeur	Justification
Format	8N1	standard
Baudrate	115 200	debug usuel
F horloge	50 MHz	générique
Ticks par bit	$50\text{e}6/115200 = 434$ (arrondi)	erreur = ? % (à calculer)
Erreur cumulée sur 10 bits	?	doit rester $\ll \frac{1}{2}$ bit

Calcul attendu : 434 ticks → baud réel 115 207, erreur ≈ 0,006 %, soit 0,06 % sur la trame :

négligeable (la limite communément admise est $\sim 2\%$). **Savoir faire ce calcul est un objectif du TP.**

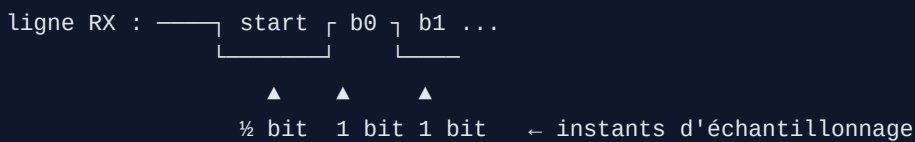
Étape 1.2 — Émetteur

Reprends `uart_tx` du TD 04 exercice 5 (ou réécris-le de mémoire, c'est mieux). Simule avec son testbench.  **Point de contrôle 1** : au chronogramme, mesure avec les curseurs de GTKWave la durée d'un bit — elle doit valoir $434 \text{ cycles} \pm 0$.

Séance 2 (3 h) — Le récepteur (le morceau noble)

Étape 2.1 — Comprendre le sur-échantillonnage

Le récepteur n'a pas l'horloge de l'émetteur : il se resynchronise sur le front descendant du start, puis échantillonne chaque bit **en son milieu** :



ligne RX : — start — b0 — b1 ...

▲ ▲ ▲

$\frac{1}{2}$ bit 1 bit 1 bit ← instants d'échantillonnage

Détection du start à mi-bit : si la ligne est encore à 0, c'est un vrai start (sinon, c'était un parasite → retour au repos). C'est un **filtre anti-glitch gratuit** — note-le dans ton compte rendu.

Étape 2.2 — Coder `uart_rx`

Squelette imposé (complète les `-- À FAIRE`) :

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity uart_rx is
    generic ( F_CLK : natural := 50_000_000; BAUD : natural := 115_200 );
    port (
        clk      : in  std_logic;
        rx       : in  std_logic;                      -- ligne série (ASYNCHRONE)
        donnee   : out std_logic_vector(7 downto 0);
        valide   : out std_logic                       -- '1' pendant 1 cycle
    );
end entity;

architecture rtl of uart_rx is
    constant TPB : natural := F_CLK / BAUD;
    type t_etat is (REPOS, VERIF_START, DONNEES, STOP);
    signal etat : t_etat := REPOS;
    signal sync0, sync1 : std_logic := '1';           -- synchroniseur 2 FF
    signal cpt : natural range 0 to TPB - 1 := 0;
    signal idx : natural range 0 to 7 := 0;
    signal shreg : std_logic_vector(7 downto 0);
begin
    process (clk)
    begin
        if rising_edge(clk) then
            sync0 <= rx;                               -- JAMAIS rx directement
            sync1 <= sync0;
            valide <= '0';                             -- impulsion par défaut

            case etat is
                when REPOS =>
                    if sync1 = '0' then                 -- front de start ?
                        cpt <= 0; etat <= VERIF_START;
                    end if;
                when VERIF_START =>
                    if cpt = TPB/2 - 1 then             -- milieu du start
                        if sync1 = '0' then             -- toujours bas : vrai start
                            cpt <= 0; idx <= 0; etat <= DONNEES;
                        else
                            etat <= REPOS;              -- parasite : on abandonne
                        end if;
                    else
                        cpt <= cpt + 1;
                    end if;
                when DONNEES =>
                    -- À FAIRE : attendre TPB cycles, échantillonner sync1
                    -- dans shreg(idx) (LSB d'abord), 8 fois
                when STOP =>
                    -- À FAIRE : attendre TPB cycles ; si sync1='1',
                    -- donnee <= shreg et valide <= '1' ; retour REPOS
            end case;
        end if;
    end process;
end rtl;

```

```
end process;
end architecture;
```

Étape 2.3 — Testbench en boucle (le juge de paix)

Le test le plus élégant : **brancher ton TX sur ton RX** et vérifier que ce qui sort est ce qui est entré — pour les 256 valeurs possibles :

```
architecture sim of tb_boucle is
    signal clk, tx_start, fil, busy, valide : std_logic := '0';
    signal d_in, d_out : std_logic_vector(7 downto 0);
begin
    emetteur : entity work.uart_tx port map (clk, tx_start, d_in, fil, busy);
    recepteur : entity work.uart_rx port map (clk, fil, d_out, valide);
    clk <= not clk after 10 ns; -- 50 MHz

    stim : process
    begin
        for i in 0 to 255 loop -- test EXHAUSTIF
            d_in <= std_logic_vector(to_unsigned(i, 8));
            wait until rising_edge(clk);
            tx_start <= '1'; wait until rising_edge(clk); tx_start <= '0';
            wait until valide = '1';
            assert d_out = d_in
                report "echec pour " & integer'image(i) severity failure;
            wait until busy = '0';
        end loop;
        report "BOUCLE OK : 256/256 octets";
        wait;
    end process;
end architecture;
```

✓ **Point de contrôle 2** : **BOUCLE OK : 256/256** . Si un octet échoue, ouvre le chronogramme à cet octet précis et compare les instants d'échantillonnage — 90 % des bugs sont un décompte décalé d'un cycle (erreur « off by one » sur **cpt = TPB - 1** vs **TPB**).

Séance 3 (3 h) — Robustesse et intégration

Étape 3.1 — Tests de robustesse (à ajouter au testbench)

1. **Glitch** : force **fil <= '0'** pendant 1 µs (< ½ bit) hors trame ; **valide** ne doit **jamais** se lever.
2. **Horloges désaccordées** : instancie l'émetteur avec **F_CLK => 50_000_000** et le récepteur avec **F_CLK => 49_000_000** (−2 %) : la boucle des 256 doit encore passer. À −5 %, observe et explique l'échec (l'erreur cumulée dépasse ½ bit au 8^e bit).
3. **Trame sans stop** (émetteur trafiqué ou stimulus manuel) : le récepteur doit revenir au repos sans se verrouiller.

Étape 3.2 — Module d'écho

Assemble un `top_echo` : tout octet reçu est réémis (RX.valide → TX.tx_start, avec gestion du cas « TX occupé » : un registre d'attente d'un octet suffit — documente ce choix). Testbench : envoie « VHDL », vérifie l'écho.

Étape 3.3 — (option carte réelle)

Basys 3 : `create_clock -period 10.0` sur l'horloge 100 MHz (adapte le generic !), broches USB-UART B18 (RX) / A18 (TX) dans le `.xdc` ; ouvre un terminal série à 115 200 : ce que tu tapes revient à l'écran — ton matériel parle au PC.

Livrables et barème

Livrable	Points
Spécification remplie avec calcul d'erreur de baudrate	/3
<code>uart_rx</code> complet : synchroniseur, vérif à mi-start, milieu de bit	/5
Boucle 256/256 verte	/4
Les 3 tests de robustesse + explication du -5 %	/4
Écho fonctionnel avec cas « TX occupé » traité et documenté	/3
Compte rendu : chronogrammes annotés (captures GTKWave)	/1
Total	/20

Ce que ce TP t'a fait pratiquer : FSM matérielles, synchronisation inter-domaines, sur-échantillonnage, testbench exhaustif et tests aux limites — le quotidien exact d'un ingénieur FPGA.